

Summe – iterativ + rekursiv

Pseudocode (IHK-Stil):

```
funktion summeIter(A: Array<Integer>) → Integer
    s = 0
    für i = 0 bis länge(A)-1
        s = s + A[i]
    return s
end funktion
```

```
funktion summeRek(A: Array<Integer>, n: Integer) → Integer
    wenn n == 0 dann return 0
    sonst return summeRek(A, n-1) + A[n-1]
end funktion
```

Java-Beispiel:

```
public class StatistikSumme {
    static int summeIter(int[] A) {
        int s = 0;
        for (int x : A) s += x;
        return s;
    }
    static int summeRek(int[] A, int n) {
        if (n == 0) return 0;
        return summeRek(A, n-1) + A[n-1];
    }
}
```

Summe – Erklärung & Beispiele

****Iterativ:**** Schleife addiert alle Elemente.

****Rekursiv:**** Summe der ersten $n-1$ Elemente + letztes Element.

****Komplexität:**** $O(n)$ Zeit; iterativ $O(1)$ Speicher, rekursiv $O(n)$ Stack.

****Prüfungstipp:**** Rekursion nur verwenden, wenn explizit gefragt (Stack!).

Array	summelter	summeRek(A, n)
[2,3,5]	10	10
[]	0	0

Durchschnitt (Mittelwert)

Pseudocode (IHK-Stil):

```
funktion durchschnitt(A: Array<Integer>) → Double
    wenn länge(A) == 0 dann return 0
    s = 0
    für i = 0 bis länge(A)-1
        s = s + A[i]
    return s / länge(A)
end funktion
```

Java-Beispiel:

```
public class StatistikDurchschnitt {
    static double durchschnitt(int[] A) {
        if (A.length == 0) return 0;
        long s = 0;
        for (int x : A) s += x;
        return (double) s / A.length;
    }
}
```

Durchschnitt – Erklärung & Beispiele

Mittelwert = Summe / Anzahl.

****Typische Fehler:**** Integer-Division (immer zu Double casten!), Leeres Array nicht beachten.

****Prüfungstipp:**** In Java bei großen Summen `long` für Summe verwenden.

Array	Summe	Anzahl	Durchschnitt
[2,3,5]	10	3	3.333...
[1,1,1,1]	4	4	1.0

Minimum & Maximum – iterativ (Min/Max) + rekursiv (Max)

Pseudocode (IHK-Stil):

```
funktion findeMin(A: Array<Integer>) → Integer
    wenn länge(A) == 0 dann return MAX_INT
    m = A[0]
    für i = 1 bis länge(A)-1
        wenn A[i] < m dann m = A[i]
    return m
end funktion
```

```
funktion findeMax(A: Array<Integer>) → Integer
    wenn länge(A) == 0 dann return MIN_INT
    m = A[0]
    für i = 1 bis länge(A)-1
        wenn A[i] > m dann m = A[i]
    return m
end funktion
```

```
funktion maxRek(A: Array<Integer>, n: Integer) → Integer
    wenn n == 1 dann return A[0]
    sonst return max( A[n-1], maxRek(A, n-1) )
end funktion
```

Java-Beispiel:

```
public class StatistikMinMax {
    static int findeMin(int[] A) {
        if (A.length == 0) return Integer.MAX_VALUE;
        int m = A[0];
        for (int i = 1; i < A.length; i++) if (A[i] < m) m = A[i];
        return m;
    }
    static int findeMax(int[] A) {
        if (A.length == 0) return Integer.MIN_VALUE;
        int m = A[0];
        for (int i = 1; i < A.length; i++) if (A[i] > m) m = A[i];
        return m;
    }
    static int maxRek(int[] A, int n) {
        if (n == 1) return A[0];
        return Math.max(A[n-1], maxRek(A, n-1));
    }
}
```

Min/Max – Erklärung & Beispiele

****Iterativ:**** Laufende Aktualisierung eines Minimums/Maximums.

****Rekursiv (Max):**** Maximum der ersten n-1 Elemente mit A[n-1] vergleichen.

****Typische Fehler:**** Leeres Array nicht behandeln; falsche Initialwerte.

****Prüfungstipp:**** Für leeres Array klare Vorgabe (z. B. MIN_INT/MAX_INT oder Exception) nennen.

Array	Min	Max	Max rekursiv
[7,2,9,1]	1	9	9
[]	MAX_INT	MIN_INT	-

Median (mit Sortierung)

Pseudocode (IHK-Stil):

```
funktion median(A: Array<Integer>) → Double
    wenn länge(A) == 0 dann return 0
    B = kopie(A)
    sortiere(B) // z.B. insertionSort
    n = länge(B)
    wenn n % 2 == 1 dann
        return B[n/2]
    sonst
        return (B[n/2 - 1] + B[n/2]) / 2.0
end funktion
```

Java-Beispiel:

```
import java.util.Arrays;

public class StatistikMedian {
    static double median(int[] A) {
        if (A.length == 0) return 0;
        int[] B = Arrays.copyOf(A, A.length);
        Arrays.sort(B);
        int n = B.length;
        if (n % 2 == 1) return B[n/2];
        else return (B[n/2 - 1] + B[n/2]) / 2.0;
    }
}
```

Median – Erklärung & Beispiele

Der Median ist der Wert in der Mitte einer sortierten Liste.
Bei gerader Anzahl: Durchschnitt der beiden mittleren Werte.

****Komplexität:**** Sortieren dominiert $\rightarrow O(n \log n)$.

****Prüfungstipp:**** Median ist robust gegenüber Ausreißern (anders als Durchschnitt).

Array	Sortiert	Median
[7,2,9,1]	[1,2,7,9]	$(2+7)/2 = 4.5$
[1,3,5]	[1,3,5]	3

Standardabweichung (Population, einfache Variante)

Pseudocode (IHK-Stil):

```
funktion standardabweichung(A: Array<Double>) → Double
    wenn länge(A) == 0 dann return 0
    // 1) Mittelwert
    sum = 0
    für i = 0 bis länge(A)-1
        sum = sum + A[i]
    mu = sum / länge(A)

    // 2) Varianz
    vSum = 0
    für i = 0 bis länge(A)-1
        diff = A[i] - mu
        vSum = vSum + diff * diff
    var = vSum / länge(A)    // Population

    return sqrt(var)
end funktion
```

Java-Beispiel:

```
public class StatistikStdAbw {
    static double standardabweichung(double[] A) {
        if (A.length == 0) return 0;
        double sum = 0;
        for (double x : A) sum += x;
        double mu = sum / A.length;
        double vSum = 0;
        for (double x : A) {
            double diff = x - mu;
            vSum += diff * diff;
        }
        double var = vSum / A.length; // Population
        return Math.sqrt(var);
    }
}
```

Standardabweichung – Erklärung & Beispiele

Misst die durchschnittliche Abweichung der Werte vom Mittelwert.

****Population (σ):**** Division durch n.

****Stichprobe (s):**** Division durch (n-1) – oft in Statistik, weniger in IHK-Algorithmen.

****Typische Fehler:**** Integer-Division; Mittelwert vor Varianz nicht berechnen; falscher Nenner.

****Prüfungstipp:**** Wenn nicht angegeben, Population (durch n) ist in Programmieraufgaben üblich.

Array	Mittelwert	Varianz (n)	StdAbw
[2,4,4,4,5,5,7,9]	5.0	4.0	2.0